Звіт до лабораторної роботи: життєвий цикл об'єкта, IDisposable

Тема: Життєвий цикл об'єкта, керовані й некеровані ресурси, інтерфейс `IDisposable`, оператор `using`, патерн Dispose (C#) **Варіант:** 7 **Виконав:** Зінчук Д. Ю.

1. Посилання на репозиторій

GitHub: https://github.com/zinchukdiipz25-wq/oop-ipz3-1-zinchuk

2. Постановка завдання

Реалізувати клас `SerialPortConnection`, що імітує з'єднання з серійним портом:

- поля: `_portName` (string), `_isPortOpen` (bool);
- метод `SendData(byte[] data)`;
- `Dispose`, який «закриває порт».

Клас має реалізовувати інтерфейс `IDisposable` за патерном Dispose (метод `Dispose()`, захищений `Dispose(bool disposing)` та фіналізатор). Потрібно продемонструвати використання через `using`, явний виклик `Dispose()` і випадок, коли `Dispose()` не викликано та об'єкт звільняє фіналізатор.

3. Код програми

```
using System;
using System.Runtime.CompilerServices;

class SerialPortConnection : IDisposable
{
    // Поля
    private string _portName;
    private bool _isPortOpen;
    private bool _disposed;

    // Властивість PortName з валідацією
    public string PortName
    {
        get { return _portName; }
        set
```

```
        {
            if (string.IsNullOrWhiteSpace(value))
                throw new ArgumentException("Назва порту не може бути
порожньою!");

            _portName = value;
        }
    }

    // Лише для читання
    public bool IsPortOpen
    {
        get { return _isPortOpen; }
    }

    // Конструктор без параметрів
    public SerialPortConnection() : this("COM1")
    {
        Console.WriteLine("Створено з'єднання за замовчуванням.");
    }

    // Параметризований конструктор
    public SerialPortConnection(string portName)
    {
        PortName = portName;
        _isPortOpen = true;

        Console.WriteLine($"Порт {PortName} відкрито.");
    }

    // Відправлення даних
    public void SendData(byte[] data)
    {
        if (_disposed)
            throw new ObjectDisposedException(nameof(SerialPortConnection));

        if (data == null || data.Length == 0)
            throw new ArgumentException("Дані не можуть бути порожніми!",
nameof(data));

        Console.WriteLine($"{PortName}: надіслано {data.Length} байт.");
    }

    // Публічний Dispose
    public void Dispose()
    {
```

```
        Dispose(true);
        GC.SuppressFinalize(this);
    }

    // Патерн Dispose
    protected virtual void Dispose(bool disposing)
    {
        if (_disposed)
            return;

        if (disposing)
        {
            // Тут звільнялися б інші керовані IDisposable-об'єкти
        }

        // Звільнення (імітованого) некерованого ресурсу
        if (_isPortOpen)
        {
            _isPortOpen = false;

            string source = disposing ? "Dispose" : "фіналізатор";
            Console.WriteLine($"Порт {_portName} закрито ({source}).");
        }

        _disposed = true;
    }

    // Фіналізатор: страховка, якщо Dispose() не викликали
    ~SerialPortConnection()
    {
        Dispose(false);
    }
}

class Program
{
    static void Main()
    {
        Console.OutputEncoding = System.Text.Encoding.UTF8;

        Console.WriteLine("=== З'єднання 1 (using) ===");
        using (var port1 = new SerialPortConnection())
        {
            port1.SendData(new byte[] { 1, 2, 3 });
        }
```

```
        Console.WriteLine();

        Console.WriteLine("=== З'єднання 2 (явний Dispose) ===");
        var port2 = new SerialPortConnection("COM3");
        try
        {
            port2.SendData(new byte[] { 10, 20 });
        }
        finally
        {
            port2.Dispose();
        }

        // Повторний виклик безпечний
        port2.Dispose();

        try
        {
            port2.SendData(new byte[] { 1 });
        }
        catch (ObjectDisposedException)
        {
            Console.WriteLine("Помилка: об'єкт уже звільнено.");
        }

        Console.WriteLine();

        Console.WriteLine("=== З'єднання 3 (без Dispose, лише фіналізатор)
===");
        CreateForgottenConnection();

        Console.WriteLine("=== Примусове збирання сміття ===");
        GC.Collect();
        GC.WaitForPendingFinalizers();

        Console.WriteLine("Програму завершено.");
    }

    // Окремий метод, щоб після виходу з нього посилання на об'єкт зникло
    [MethodImpl(MethodImplOptions.NoInlining)]
    static void CreateForgottenConnection()
    {
        var port3 = new SerialPortConnection("COM5");
        port3.SendData(new byte[] { 255 });
    }
}
```

4. Опис класу

Клас `SerialPortConnection` має поля `_portName`, `_isPortOpen` та службове поле `_disposed`, яке запобігає повторному звільненню. Властивість `PortName` перевіряє значення: порожня назва викликає `ArgumentException`. Властивість `IsPortOpen` доступна лише для читання.

Конструктори:

- `SerialPortConnection()` через `this("COM1")` передає керування параметризованому конструктору;
- `SerialPortConnection(string portName)` встановлює назву через властивість (тобто з валідацією) і «відкриває порт».

Метод `SendData(byte[] data)` перевіряє, що об'єкт ще не звільнено (інакше `ObjectDisposedException`) і що дані не порожні, після чого виводить кількість надісланих байтів.

Реалізація патерну Dispose:

- `Dispose()` викликає `Dispose(true)` і `GC.SuppressFinalize(this)`, щоб збирач сміття не викликав фіналізатор зайвий раз;
- `Dispose(bool disposing)` закриває порт один раз. Параметр `disposing` показує, чи метод викликано явно (тоді можна звільняти й керовані об'єкти), чи з фіналізатора (тоді керованих об'єктів чіпати не можна);
- `~SerialPortConnection()` викликає `Dispose(false)` як страховку на випадок, якщо програміст забув викликати `Dispose()`.

5. Результат виконання програми

Нижче наведено очікуваний вивід консолі.

```
=== З'єднання 1 (using) ===
Порт COM1 відкрито.
Створено з'єднання за замовчуванням.
COM1: надіслано 3 байт.
Порт COM1 закрито (Dispose).

=== З'єднання 2 (явний Dispose) ===
Порт COM3 відкрито.
COM3: надіслано 2 байт.
Порт COM3 закрито (Dispose).
Помилка: об'єкт уже звільнено.

=== З'єднання 3 (без Dispose, лише фіналізатор) ===
Порт COM5 відкрито.
COM5: надіслано 1 байт.
=== Примусове збирання сміття ===
Порт COM5 закрито (фіналізатор).
Програму завершено.
```

Місце для скриншота консолі: вставте власний скриншот після запуску програми.

6. Висновок

У роботі реалізовано клас `SerialPortConnection` з патерном Dispose. Життєвий цикл об'єкта складається зі створення (виділення пам'яті в купі та виклик ланцюга конструкторів), використання через посилання, втрати останнього посилання, коли об'єкт стає кандидатом на збирання сміття, і необов'язкової фіналізації.

Демонстрація показала різницю між детермінованим і недетермінованим звільненням. Коли використано `using` або явний `Dispose()`, порт закривається у передбачуваний момент (на виході з блоку), а `GC.SuppressFinalize` не дозволяє виконатися фіналізатору. Коли `Dispose()` забули викликати, порт закрився лише після `GC.Collect()` через фіналізатор, а момент цього закриття визначає збирач сміття, а не програміст.

Тому для некерованих ресурсів (файли, з'єднання з БД, порти) слід завжди звільняти їх через `IDisposable` та `using`, а фіналізатор залишати лише як запасний механізм. Примусовий виклик `GC.Collect()` використано виключно для навчальної демонстрації.